

EXPERIMENT 5: FILTERING USING AVERAGING FILTER, UNSHARP MASKING AND HIGH BOOST FILTERING

AIM:

To perform filtering using averaging filter, unsharp masking and high boost filtering using python in google colab.

MATERIALS REQUIRED:

Google colab software (python)

CODE:

1)Averaging Filter

Theory:

-Neighborhood processing in spatial domain: Here, to modify one pixel, we consider values of the immediate neighboring pixels also. For this purpose, 3X3, 5X5, or 7X7 neighborhood mask can be considered. An example of a 3X3 mask is shown below.

$f(x-1, y-1)$ $f(x-1, y)$ $f(x-1, y+1)$

$f(x, y-1)$ $f(x, y)$ $f(x, y+1)$

$f(x+1, y-1)$ $f(x+1, y)$ $f(x+1, y+1)$

-Low Pass filtering: It is also known as the smoothing filter. It removes the high-frequency content from the image. It is also used to blur an image. A low pass averaging filter mask is as shown.

$1/9$ $1/9$ $1/9$

$1/9$ $1/9$ $1/9$

$1/9$ $1/9$ $1/9$

-High Pass Filtering: It eliminates low-frequency regions while retaining or enhancing the high-frequency components. A high pass filtering mask is as shown.

$-1/9$ $-1/9$ $-1/9$

$-1/9$ $8/9$ $-1/9$

$-1/9$ $-1/9$ $-1/9$

Code:

```
import matplotlib.pyplot as plt
from PIL import Image

# Open the original and blurred images
original_img = Image.open('foot Xray.jpg')
blurred_img = Image.open('blurred.jpg')

# Display original and blurred images
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
axes[0].imshow(original_img, cmap='gray')
axes[0].set_title('Original Image')
axes[0].axis('off')

axes[1].imshow(blurred_img, cmap='gray')
axes[1].set_title('Blurred Image')
axes[1].axis('off')

plt.tight_layout()
plt.show()
```

OUTPUT:



2) Unsharp Masking

Theory:

PIL is the Python Imaging Library which provides the python interpreter with image editing capabilities. The ImageFilter module contains definitions for a pre-defined set of filters, which can be used with the Image.filter() method.

PIL.ImageFilter.UnsharpMask() method applies the Unsharp mask filter to the input image.

Syntax: PIL.ImageFilter.UnsharpMask(radius=2, percent=150, threshold=3)

radius: Blur Radius

percent: Unsharp strength, in percent

threshold: Threshold controls the minimum brightness change that will be sharpened

Code:

```
from PIL import Image, ImageFilter
```

```
import matplotlib.pyplot as plt
```

```
# Open the image
```

```
img = Image.open('foot Xray.jpg')
```

```
# Apply UnsharpMask filter
```

```
# radius: blur radius (float or int)
```

```
# percent: unsharp mask strength, in percent (float or int)
```

```
# threshold: threshold for sharpening (float or int)
```

```
sharpened_img = img.filter(ImageFilter.UnsharpMask(radius=2, percent=150,  
threshold=3))
```

```
# Display original and sharpened images
```

```
fig, axes = plt.subplots(1, 2, figsize=(10, 5))
```

```
axes[0].imshow(img, cmap='gray')
```

```
axes[0].set_title('Original Image')
```

```
axes[0].axis('off')
```

```
axes[1].imshow(sharpened_img, cmap='gray')
```

```
axes[1].set_title('Sharpened Image (Unsharp Mask)')
```

```
axes[1].axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



3)High Boost Filtering

Theory:

To apply the High - Boost Filter to an image we have to follow certain steps. Here are the steps to be followed –

- **Smooth the image:** Firstly we have to smooth the image by using a low pass filter such as Guassian filter to extract low-frequency components.
- **Subtract the smoothed image:** Next we have to subtract the smoothed image from the original image to extract the high-frequency components.
- **Add back the original image:** Finally we have to add back the original image multiplied by the amplification factor to retain and amplify the low-frequency content.

-Basic High-Boost Filtering Example

Following is the example of the basic high boost filter applied to the given input image by using the function **scipy.ndimage.guassian()** –

Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy import ndimage
```

```
from skimage import data, color, io
```

```
# Load a sample image (e.g., astronaut image from skimage)
```

```
# The original line was: image = color.rgb2gray(foot Xray.jpg)
```

```
# This is fixed by using io.imread to load the image and converting it to grayscale
```

```
image = io.imread('foot Xray.jpg', as_gray=True) # Load as grayscale and convert to float [0, 1]
```

```
# Define the amplification factor
```

```
A = 1.5 # Adjust this value for more or less sharpening
```

```
# Create a smoothed version of the image using Gaussian filter
```

```
smoothed = ndimage.gaussian_filter(image, sigma=3)
```

```
# Compute the high-boost filtered image
```

```
high_boost = A * image - smoothed
```

```
# Plot the original, smoothed, and high-boost filtered images
```

```
plt.figure(figsize=(15, 5))
```

```
plt.subplot(1, 3, 1)
```

```
plt.title("Original Image")
```

```
plt.imshow(image, cmap='gray')
```

```
plt.axis('off')
```

```
plt.subplot(1, 3, 2)
```

```
plt.title("Smoothed Image (Low-Pass Filter)")
```

```
plt.imshow(smoothed, cmap='gray')
```

```
plt.axis('off')
```

```
plt.subplot(1, 3, 3)
```

```
plt.title(f"High-Boost Filtered Image (A = {A})")
```

```
plt.imshow(high_boost, cmap='gray')
```

```
plt.axis('off')
```

```
plt.show()
```

OUTPUT:



-Varying Amplification Factor A

The high-boost filter is a popular image sharpening technique that enhances details by amplifying high-frequency components while retaining some of the original image's low-frequency content. The amplification factor **A** in the high-boost filter plays a key role in determining the intensity of the sharpening effect. The High boost filter formula in terms of Amplification factor can be given as follows –

High-Boost Image = $A \cdot I - \text{Blurred Image} = I + (A - 1) \cdot \text{Mask}$

Where –

- **I:** Original Image
- **BlurredImage:** Result of applying a smoothing (low-pass) filter to I.
- **A:** Amplification factor which is typically $A \geq 1$.
- **Mask:** Difference between the original image and the blurred image ($I - \text{BlurredImage}$).

For $A = 1$, the result is equivalent to standard image sharpening. For $A > 1$, the high-boost effect becomes stronger.

Below is an example of applying a high-boost filter to an image with different amplification factors.–

Code:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy import ndimage
from skimage import data, color, io
```

```
# Load and preprocess the image
image = io.imread(' foot Xray.jpg', as_gray=True) # Load as grayscale

# Define a Gaussian blur for low-pass filtering
blurred = ndimage.gaussian_filter(image, sigma=2)

# Compute the mask (high-frequency components)
mask = image - blurred

# Apply high-boost filtering with different A values
A_values = [1, 1.5, 2, 3] # Amplification factors
high_boost_images = [image + (A - 1) * mask for A in A_values]

# Plot original, mask, and high-boost images
plt.figure(figsize=(15, 8))

# Original image
plt.subplot(2, len(A_values) + 1, 1)
plt.title("Original Image")
plt.imshow(image, cmap='gray')
plt.axis('off')

# Mask
plt.subplot(2, len(A_values) + 1, 2)
plt.title("Mask (High-Freq Components)")
plt.imshow(mask, cmap='gray')
plt.axis('off')

# High-boost images
```

```
for i, (A, hb_image) in enumerate(zip(A_values, high_boost_images), start=3):
```

```
    plt.subplot(2, len(A_values) + 1, i)
```

```
    plt.title(f"High-Boost A={A}")
```

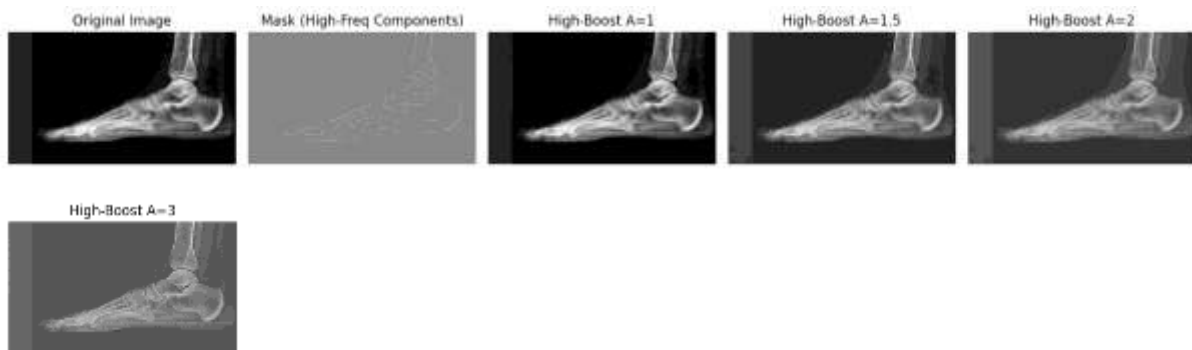
```
    plt.imshow(hb_image, cmap='gray')
```

```
    plt.axis('off')
```

```
plt.tight_layout()
```

```
plt.show()
```

OUTPUT:



-Comparing High-Pass & High-Boost Filters

The high-pass filter and high-boost filter are closely related but they serve slightly different purpose. The High pass filter enhances high-frequency components by removing low-frequency components whereas the High Boost filter enhances high-frequency components while retaining some of the original image's low-frequency content, controlled by an amplification factor A .

Following example highlights the difference between a high-pass filter ($A=1$) and a high-boost filter ($A > 1$).

Code:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
from scipy import ndimage
```

```
from skimage import data, color, io
```

```
# Load and preprocess the image
image = io.imread('foot Xray', as_gray=True) # Load as grayscale

# Apply a Gaussian blur for low-pass filtering
blurred = ndimage.gaussian_filter(image, sigma=2)

# Compute the high-pass filter result
high_pass = image - blurred

# Compute the high-boost filter result with varying amplification factors
A = 2 # Amplification factor for high-boost
high_boost = image + (A - 1) * high_pass

# Plot original, high-pass, and high-boost images
plt.figure(figsize=(15, 5))

plt.subplot(1, 3, 1)
plt.title("Original Image")
plt.imshow(image, cmap='gray')
plt.axis('off')

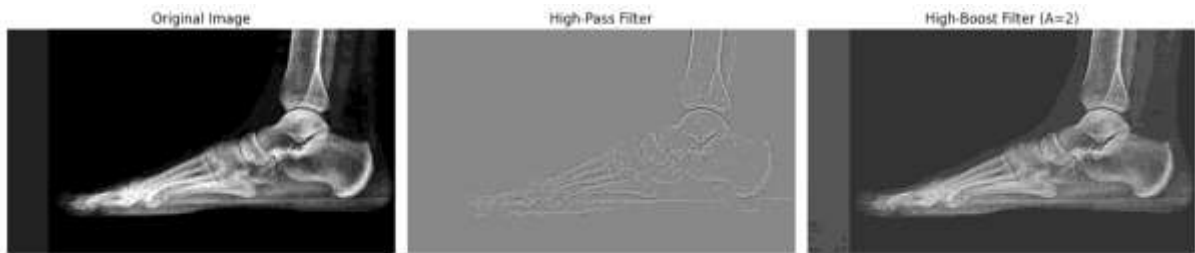
plt.subplot(1, 3, 2)
plt.title("High-Pass Filter")
plt.imshow(high_pass, cmap='gray')
plt.axis('off')

plt.subplot(1, 3, 3)
plt.title(f"High-Boost Filter (A={A})")
```

```
plt.imshow(high_boost, cmap='gray')  
plt.axis('off')
```

```
plt.tight_layout()  
plt.show()
```

OUTPUT:



RESULT:

Thus the filtering using averaging filter, unsharp masking and high boost filtering were performed by google colab.

EXPERIMENT 6: COLOR IMAGE PROCESSING

AIM:

To perform color image processing using python in google colab.

MATERIALS REQUIRED:

Google colab software (python)

CODE:

- Color Identification Using OpenCV

A color histogram is a representation of the distribution of colors in an image using the number of pixels present in a particular color and presenting them in a histogram.

Let us see steps how we can perform color identification in images using Python and OpenCV.

Step 1: Importing the Required Libraries

To convert images between the BGR and RGB color spaces, we need to import the required libraries. We will be using the following libraries ?

- **OpenCV** ? To read and manipulate images.
- **Matplotlib** ? To display images.

```
import cv2
```

```
import numpy as np
```

Step 2: Load the Image and Convert it to the HSV Color Space

Now we will load the image and convert it into the HSV color space.

```
img = cv2.imread('brain MRI.jpg')
```

```
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
```

Step 3: Define the Color Range

We need to define the lower and upper bounds of the color range we want to detect. We can use the `inRange()` function of OpenCV to extract the pixels within the color range.

```
lower_range = (0, 50, 50) # lower range of red color in HSV
```

```
upper_range = (10, 255, 255) # upper range of red color in HSV  
mask = cv2.inRange(hsv_image, lower_range, upper_range)
```

Step 4: Apply a Mask to the Image

We can apply the mask we obtained in the previous step to the original image to extract only the pixels within the color range.

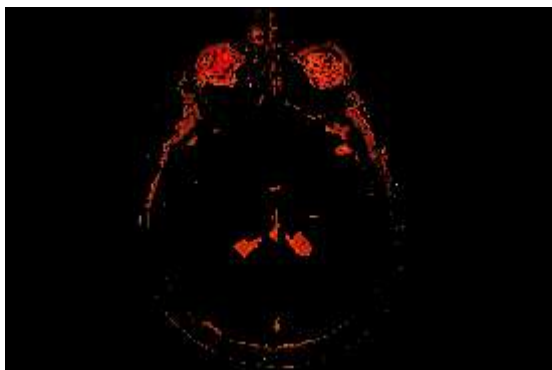
```
color_image = cv2.bitwise_and(image, image, mask=mask)
```

Step 5: Display the Color Image

Finally, we can display the color image that contains only the pixels within the color range we specified.

```
# Display the color image  
cv2.imshow('Color Image', color_image)  
cv2.waitKey(0)  
cv2.destroyAllWindows()
```

OUTPUT:



Code:

```
import cv2  
import numpy as np  
from google.colab.patches import cv2_imshow  
  
img = cv2.imread('brain MRI.jpg')
```

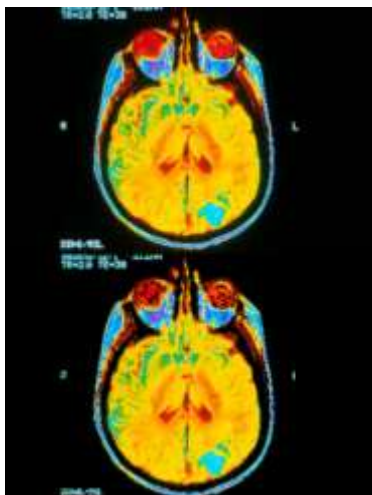
```
cv2_imshow(img)
cv2.waitKey(0)
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)

# lower range of red color in HSV
lower_range = (0, 50, 50)

# upper range of red color in HSV
upper_range = (150, 255, 255)
mask = cv2.inRange(hsv_img, lower_range, upper_range)
color_image = cv2.bitwise_and(img, img, mask=mask)

# Display the color of the image
cv2_imshow(color_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



- Detecting Different Colors

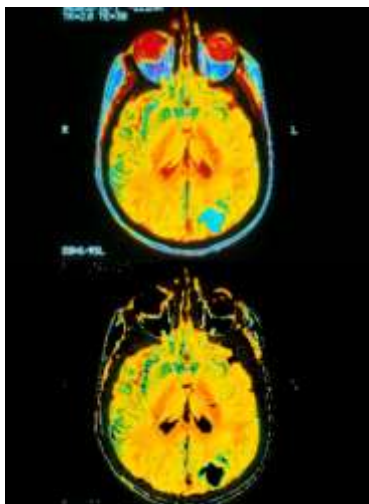
In the above code, we have identified the pixels within the red color range. We can modify the lower and upper ranges to detect different colors. For example, If you need to detect green color, we can set the lower and upper ranges as follows ?

Code:

```
import cv2
import numpy as np
from google.colab.patches import cv2_imshow

img = cv2.imread('brain MRI.jpg')
cv2_imshow(img)
cv2.waitKey(0)
hsv_img = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
lower_range = (12, 25, 25) # lower range of green color in HSV
upper_range = (86, 255, 255) # upper range of green color in HSV
mask = cv2.inRange(hsv_img, lower_range, upper_range)
color_image = cv2.bitwise_and(img, img, mask=mask)
cv2_imshow(color_image)
cv2.waitKey(0)
cv2.destroyAllWindows()
```

OUTPUT:



RESULT:

Thus the color image processing were performed by google colab.